

Capítulo 02: Métodos de Otimização em Biologia de Sistemas

Levenberg-Marquardt, Evolução Diferencial e Algoritmos Genéticos

Ney Lemke

DBF-Unesp

23 de junho de 2026

Sumário

- 1 Introdução à Otimização
- 2 Levenberg-Marquardt
- 3 Evolução Diferencial
- 4 Algoritmos Genéticos
- 5 Comparação e Recomendações

O que é otimização?

Definição

Otimização é o processo de encontrar a melhor solução possível para um problema, dado um conjunto de restrições e uma função objetivo que mede a qualidade de cada solução.

Elementos fundamentais:

- **Função objetivo:** mede a qualidade ($f(x)$)
- **Variáveis de decisão:** parâmetros a otimizar
- **Restrições:** limites e condições
- **Espaço de busca:** região válida de soluções

Analogia Biológica

A otimização em biologia de sistemas é como a seleção natural: encontrar a “melhor” configuração de parâmetros que maximiza a aptidão do sistema biológico.

Classificação dos métodos de otimização (1/2)

Métodos Determinísticos

- Gradiente descendente
- Newton, Quasi-Newton
- Levenberg-Marquardt
- Programação linear/não-linear

Características:

- Resultados reprodutíveis
- Rápidos para funções suaves
- Sensíveis a mínimos locais

Classificação dos métodos de otimização (2/2)

Métodos Estocásticos

- Algoritmos Genéticos
- Evolução Diferencial
- Recozimento Simulado
- Enxame de Partículas

Características:

- Exploram espaço globalmente
- Menos sensíveis a mínimos locais
- Mais lentos, mais robustos

Contexto Real

Geralmente trabalhamos com funções objetivo:

- Não-lineares e multimodais
- Com ruído experimental
- Parâmetros correlacionados

Frequentemente precisamos de métodos híbridos ou estocásticos.

Aplicações em Biologia de Sistemas (1/2)

Ajuste de parâmetros em modelos

- Cinética enzimática (Michaelis-Menten)
- Redes regulatórias gênicas
- Vias de sinalização
- Modelos metabólicos (fluxos)

Problemas típicos

- Muitos parâmetros (10-1000+)
- Dados experimentais com ruído
- Múltiplos mínimos locais
- Avaliações de função custosas

Exemplo: Estimação de K_m e V_{max}

Minimizar a soma dos quadrados dos resíduos entre os dados experimentais e o modelo de Michaelis-Menten:

$$S(\theta) = \sum_{i=1}^n (y_i - f(x_i, \theta))^2$$

onde $f(x, \theta) = \frac{V_{max} \cdot x}{K_m + x}$

Levenberg-Marquardt: Fundamentos

História

Desenvolvido por Kenneth Levenberg (1944) e Donald Marquardt (1963), é o método padrão para ajuste de mínimos quadrados não-lineares.

Ideia Central

Combina duas estratégias:

1. **Método de Gauss-Newton**: eficiente quando próximo da solução
2. **Gradiente Descendente**: confiável quando longe da solução

Um parâmetro λ controla a interpolação entre os dois métodos.

Quando usar?

- Ajuste de modelos a dados experimentais
- Minimização de soma de quadrados de resíduos
- Problemas de regressão não-linear
- Calibração de parâmetros em sistemas biológicos

Atualização de parâmetros

$$p_{k+1} = p_k + (J^T J + \lambda I)^{-1} J^T r$$

- p : vetor de parâmetros
- J : matriz Jacobiana
- r : vetor de resíduos
- λ : parâmetro de amortecimento

Adaptação de λ

- λ alto $\rightarrow$ gradiente descendente
- λ baixo $\rightarrow$ Gauss-Newton

Se o erro diminui: $\lambda \leftarrow \lambda/\nu$
Se o erro aumenta: $\lambda \leftarrow \lambda \cdot \nu$
Geralmente $\nu = 10$

Algoritmo Levenberg-Marquardt

Passos do algoritmo

1. Inicializar p_0 e $\lambda > 0$
2. Calcular resíduos $r(p_k)$
3. Calcular Jacobiana $J(p_k)$
4. Resolver sistema: $(J^T J + \lambda I) \Delta = J^T r$
5. Propor $p_{novo} = p_k + \Delta$
6. Calcular $S(p_{novo})$
7. Se $S(p_{novo}) < S(p_k)$:
 - Aceitar: $p_{k+1} = p_{novo}$
 - Reduzir: $\lambda = \lambda/\nu$
8. Senão: $\lambda = \lambda \cdot \nu$ e repetir passo 4
9. Repetir até convergência

Implementação em Python (scipy) (1/3)

```
1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 # Modelo de Michaelis-Menten
5 def michaelis_menten(x, Vmax, Km):
6     return Vmax * x / (Km + x)
7
8 # Dados experimentais
9 x_data = np.array([0.1, 0.5, 1.0, 2.0, 5.0])
10 y_data = np.array([0.5, 2.0, 3.2, 4.5, 5.8])
11 y_errors = np.array([0.1, 0.2, 0.3, 0.2, 0.3])
```

Listing 1: Ajuste Michaelis-Menten (Definição e Dados)

Implementação em Python (scipy) (2/3)

```
1 # Ajuste com Levenberg-Marquardt
2 params, cov = curve_fit(
3     michaelis_menten,
4     x_data, y_data,
5     p0=[6.0, 1.0], # chutes iniciais
6     sigma=y_errors,
7     method='lm' # Levenberg-Marquardt
8 )
9
10 Vmax, Km = params
11 print(f"Vmax = {Vmax:.3f}")
12 print(f"Km = {Km:.3f}")
```

Listing 2: Ajuste Michaelis-Menten (Fitting e Resultados)

Implementação em Python (scipy) (3/3)

Parâmetros importantes

- `p0`: chute inicial dos parâmetros
- `bounds`: limites inferior/superior
- `sigma`: incertezas dos dados
- `maxfev`: máximo de avaliações

Vantagens do método

- Muito eficiente para mínimos quadrados
- Converge rapidamente perto do ótimo
- Implementado em `numpy`, `scipy`, `MATLAB`

Vantagens e Limitações do Levenberg-Marquardt

Vantagens

- **Rápido**: converge em poucas iterações
- **Robusto**: funciona bem com chutes ruins
- **Eficiente**: usa informação do gradiente
- **Padrão**: disponível em todos os pacotes
- **Robusto a ruído**: bom para dados experimentais

Limitações

- **Mínimo local**: pode ficar preso
- **Escala**: sensível à escala dos parâmetros
- **Jacobiana**: precisa calcular ou aproximar
- **Não-global**: não garante solução ótima global

Quando NÃO usar Levenberg-Marquardt

Evolução Diferencial: Fundamentos (1/2)

Introdução

Método de otimização estocástico proposto por Rainer Storn e Kenneth Price (1997), particularmente eficaz para otimização global de funções não-lineares e não-diferenciáveis.

Características principais

- Baseado em evolução population-based
- Usa operações vetoriais diferenciadas
- Não requer cálculo de derivadas
- Bom para otimização global
- Paralelizável naturalmente

Aplicações em biologia

- Ajuste de modelos cinéticos
- Otimização de fluxos
- Parameter estimation (SBML)
- Docking molecular

Evolução Diferencial: Fundamentos (2/2)

Inspiração biológica

Semelhante aos Algoritmos Genéticos, mas usa vetores diferença para guiar a busca, simulando a variação genética diferencial em populações.

A principal inovação é a mutação baseada na diferença entre indivíduos da própria população.

Operadores Fundamentais (1/2)

Mutação Diferencial

Para cada vetor x_i na geração g :

$$v_i = x_{r_1} + F \cdot (x_{r_2} - x_{r_3})$$

- x_{r_1, r_2, r_3} : vetores aleatórios
- F : fator de escala (tipicamente 0.5-1.0)
- $r_1 \neq r_2 \neq r_3 \neq i$

Operadores Fundamentais (2/2)

Cruzamento (Crossover)

$$u_j = \begin{cases} v_{i,j} & \text{se } rand \leq CR \\ x_{i,j} & \text{caso contrário} \end{cases}$$

- CR : taxa de cruzamento (0.1-1.0)

Seleção

$$x_i^{g+1} = \begin{cases} u_i & \text{se } f(u_i) \leq f(x_i^g) \\ x_i^g & \text{caso contrário} \end{cases}$$

Algoritmo da Evolução Diferencial (1/2)

Pseudocódigo

1. **Inicializar** população aleatória $x_i, i = 1, \dots, NP$
2. **Avaliar** aptidão de todos os indivíduos
3. **Repetir** até convergência ou máximo de gerações:
 - 3.1 Para cada indivíduo i :
 - 3.1.1 Selecionar 3 indivíduos aleatórios $r_1, r_2, r_3 \neq i$
 - 3.1.2 Criar vetor mutante v_i
 - 3.1.3 Criar vetor de teste u_i via cruzamento
 - 3.1.4 Selecionar: $x_i^{g+1} = u_i$ ou x_i^g
 - 3.2 Avaliar nova população
4. **Retornar** melhor solução encontrada

Algoritmo da Evolução Diferencial (2/2)

Parâmetros principais

- NP : tamanho da população (tipicamente $5-10 \times D$)
- F : fator de escala (0.5-1.0)
- CR : taxa de cruzamento (0.7-0.9 é comum)
- G_{max} : máximo de gerações

A Evolução Diferencial é robusta e requer poucos parâmetros para ajuste.

Implementação em Python (scipy) (1/3)

```
1 import numpy as np
2 from scipy.optimize import differential_evolution
3
4 # Funcao objetivo: Rosenbrock em 10D
5 def rosenbrock(x):
6     return sum(100 * (x[1:] - x[:-1])**2)**2 +
7             (1 - x[:-1])**2)
8
9 # Limites para cada variavel
10 bounds = [(-5, 5)] * 10
```

Listing 3: Evolução Diferencial (Definição)

Implementação em Python (scipy) (2/3)

```
1  # Otimizacao com Evolucao Diferencial
2  result = differential_evolution(
3      rosenbrock,
4      bounds,
5      strategy='best1bin',
6      maxiter=1000,
7      popsize=15,
8      tol=1e-7,
9      mutation=(0.5, 1),
10     recombination=0.7,
11     seed=None,
12     polish=True  # refinamento final
13 )
14
15 print(f"Optimo encontrado: {result.x}")
16 print(f"Valor da funcao: {result.fun:.6f}")
17 print(f"Numero de avaliacoes: {result.nfev}")
18 print(f"Convergiu: {result.success}")
```

Listing 4: Evolução Diferencial (Otimização)

Implementação em Python (scipy) (3/3)

Parâmetros do scipy

- `bounds`: limites de cada variável
- `strategy`: estratégia de mutação
- `popsize`: tamanho da população
- `maxiter`: máximo de iterações
- `tol`: tolerância de convergência
- `polish`: refina com L-BFGS-B

Estratégias comuns

- `best1bin`: melhor + 1 diferença
- `rand1bin`: rand + 1 diferença
- `best2bin`: melhor + 2 diferenças

Estratégias de Mutação (1/2)

Estratégia	Equação	Quando usar
best1bin	$v = x_{best} + F(x_{r1} - x_{r2})$	Convergência rápida, bom para problemas unimodais
rand1bin	$v = x_{r1} + F(x_{r2} - x_{r3})$	Explora mais, melhor para multimodais
best2bin	$v = x_{best} + F(x_{r1} + x_{r2} - x_{r3} - x_{r4})$	Mais exploratório
rand2bin	$v = x_{r1} + F(x_{r2} + x_{r3} - x_{r4} - x_{r5})$	Maior diversidade
current-to-best	$v = x_i + F(x_{best} - x_i) + F(x_{r1} - x_{r2})$	Direciona para melhor

Estratégias de Mutação (2/2)

Escolha de estratégia

- **best1bin**: padrão, funciona bem na maioria dos casos
- **rand1bin**: melhor para problemas multimodais
- **rand2bin**: para alta dimensionalidade ($D > 30$)
- Teste diferentes estratégias para seu problema específico

Aplicação: Ajuste de Parâmetros (1/2)

Exemplo: Modelo de repressilator

O repressilator é um circuito sintético com 3 genes que se reprimem mutuamente. O ajuste de parâmetros cinéticos é crucial para reproduzir as oscilações observadas.

Função objetivo

$$S(\theta) = \sum_{i=1}^n (y_i^{exp} - y_i^{sim}(\theta))^2$$

Por que usar DE?

- Múltiplos mínimos locais
- Parâmetros com escalas variadas
- Simulação custosa (ODE solver)
- Necessidade de robustez

Aplicação: Ajuste de Parâmetros (2/2)

Dicas práticas

- Use `polish=True` para refinar a solução
- Escalone os parâmetros se tiver escalas muito diferentes
- Considere restrições dos parâmetros
- Paralelize as avaliações se possível

A Evolução Diferencial é frequentemente o método de escolha na comunidade de Modelagem em Biologia de Sistemas (ex: Copasi, AMIGO2).

Algoritmos Genéticos: Fundamentos (1/2)

Introdução

Método de otimização inspirado na evolução natural, proposto por John Holland (1975). Utiliza operadores de seleção, cruzamento e mutação sobre uma população de soluções candidatas.

Analogia Biológica

- **Indivíduo** = solução
- **Cromossomo** = representação
- **Aptidão** = qualidade
- **Cruzamento** = recombinação

Características

- População de soluções
- Busca estocástica
- Global e paralelizável

O Processo Evolutivo

1. **Gerar** população inicial
2. **Avaliar** indivíduos (Função de aptidão)
3. **Selecionar** os melhores (reprodutores)
4. **Gerar descendentes** via cruzamento e mutação
5. **Substituir** população antiga

GAs são ideais para problemas onde o espaço de busca é complexo e desconhecido.

Representação Cromossomial (1/2)

Codificação Binária

- Cada gene: 0 ou 1
- Cromossomo: string de bits
- Exemplo: 10110101 (8 bits)
- Transformação para valores reais:

$$x = x_{min} + \frac{b}{2^n - 1}(x_{max} - x_{min})$$

Codificação Real

- Valores contínuos diretamente
- Mais natural para otimização numérica
- Evita erro de discretização

Representação Cromossomial (2/2)

Exemplo em Biologia

Parâmetros cinéticos de um modelo de Michaelis-Menten:

$$\left[\underbrace{V_{max}}_{genes[0]}, \underbrace{K_m}_{genes[1]}, \underbrace{n}_{genes[2]}, \dots \right]$$

Codificação Mista

- Genes discretos (seleção de rota)
- Genes contínuos (parâmetros)
- Gerações combinadas

Operadores Genéticos (1/2)

Seleção

Tipos principais:

- **Roleta:** probabilidade proporcional à aptidão
- **Torneio:** k indivíduos competem
- **Elitismo:** melhores passam direto
- **Ranking:** baseada em posição

Cruzamento (Crossover)

- **Ponto único:** corta em um ponto
- **Dois pontos:** corta em dois pontos
- **Uniforme:** cada gene independently
- **Aritmético:** média ponderada

Operadores Genéticos (2/2)

Mutação

- **Bit flip:** inverte bit (binário)
- **Gaussian:** adiciona ruído (real)
- **Boundary:** substitui por limite
- **Non-uniform:** reduz com gerações

Taxas típicas

- Cruzamento: 0.6 – 0.9
- Mutação: 0.01 – 0.1
- Elitismo: 1 – 5 melhores

Algoritmo Genético Padrão (1/2)

Pseudocódigo

1. **Inicializar** população aleatória P de tamanho N
2. **Avaliar** aptidão de todos os indivíduos
3. **Repetir** até critério de parada:
 - 3.1 **Seleção**: pais para reprodução
 - 3.2 **Cruzamento**: criar descendentes
 - 3.3 **Mutação**: alterar genes
 - 3.4 **Avaliação**: calcular aptidão
 - 3.5 **Sobrevivência**: nova geração

Algoritmo Genético Padrão (2/2)

Critérios de parada

- Máximo de gerações
- Convergência da população
- Sem melhoria por n gerações

Pressão seletiva

- Muito alta: converge rápido, pode falhar
- Muito baixa: busca aleatória
- Equilíbrio necessário

Implementação com DEAP (1/2)

```
1 from deap import base, creator, tools, algorithms
2 import random
3
4 # Funcao de aptidao (minimizar)
5 creator.create("FitnessMin",
6               base.Fitness, weights=(-1.0,))
7 creator.create("Individual", list,
8               fitness=creator.FitnessMin)
9
10 # Toolbox
11 toolbox = base.Toolbox()
12 toolbox.register("attr_float", random.uniform, -10, 10)
13 toolbox.register("individual", tools.initRepeat,
14                 creator.Individual, toolbox.attr_float, n=10)
15 toolbox.register("population", tools.initRepeat,
16                 list, toolbox.individual)
17
18 # Funcao objetivo
19 def rosenbrock(individual):
20     return sum(100*(x[1:]-x[:-1]**2)**2 +
21               (1-x[:-1])**2 for x in [individual]),
```

Implementação com DEAP (2/2)

```
1 toolbox.register("evaluate", rosenbrock)
2 toolbox.register("mate", tools.cxSimulatedBinary,
3     eta=20, indpb=0.5)
4 toolbox.register("mutate", tools.mutPolynomialBounded,
5     eta=20, low=-10, up=10, indpb=0.1)
6 toolbox.register("select", tools.selTournament,
7     tournsize=3)
8
9 # Executar
10 population = toolbox.population(n=100)
11 algorithms.eaSimple(population, toolbox,
12     cxpb=0.8, mutpb=0.2, ngen=500, verbose=True)
```

Destaques do DEAP

Facilita representações complexas e paralelização.

Exemplo: Otimização de Redes Metabólicas (1/2)

Problema e Formulação

Encontrar fluxos metabólicos que maximizam a produção de um metabólito.

- Variáveis: fluxos v_i
- Restrições: $S \cdot v = 0$
- Função objetivo: maximizar produto
- Limites: $0 \leq v_i \leq v_i^{max}$

Exemplo: Otimização de Redes Metabólicas (2/2)

Por que GA?

- Não-linearidades e restrições
- Múltiplos mínimos locais
- Trade-offs biológicos

Extensões para biologia

- **MOGA**: multi-objectivo
- **Hybrid**: GA + gradiente local

GA vs. Evolução Diferencial: Comparação

Característica	GA	ED
Operadores	Seleção, cruzamento, mutação	Mutação diferencial + cruzamento
Representação	Binária ou real	Tipicamente real
Parâmetros	Muitos (taxas, métodos)	Poucos (F , CR)
Convergência	Mais lenta	Mais rápida
Exploração	Bom	Muito bom
Exploitação	Moderado	Bom
Implementação	Mais complexa	Mais simples

Quando usar cada um

Critérios de escolha

- **GA:** problemas com representações mistas, otimização combinatória
- **DE:** otimização numérica contínua, problemas com restrições
- **Recomendação:** teste ambos, use o que funcionar melhor para seu problema

Qual método escolher? (1/2)

Levenberg-Marquardt

- Use quando: gradientes bem-definidos, mínimos quadrados
- Vantagens: rápido, preciso, eficiente

Evolução Diferencial

- Use quando: otimização global
- Vantagens: robusto, poucos parâmetros

Qual método escolher? (2/2)

Algoritmos Genéticos

- Use quando: problemas complexos, multi-objetivo
- Vantagens: flexível, robusto, paralelo
- Limitações: muitos hiperparâmetros

Sempre valide sua solução com múltiplos pontos iniciais.

Estratégias Híbridas (1/2)

GA/DE + Refino Local

1. Usar método global (GA/DE) para explorar o espaço
2. Quando próximo do ótimo, usar Levenberg-Marquardt
3. Combina robustez global com precisão local

Algoritmo Híbrido Típico

- `differential_evolution` com `polish=True`
- Usa L-BFGS-B como refino final
- Implementado no `scipy`!

Estratégias avançadas

- **Memético:** GA + busca local em cada indivíduo
- **Multi-nicho:** múltiplas subpopulações
- **Co-evolução:** populações que competem/cooperam
- **Adaptive:** parâmetros que evoluem

Boa prática

Sempre valide sua solução: teste com múltiplos pontos iniciais, verifique sensibilidade, e compare com literatura.

O que aprendemos

- **Levenberg-Marquardt**: eficiente para mínimos quadrados e ajuste local.
- **Evolução Diferencial**: robusto para otimização global.
- **Algoritmos Genéticos**: flexíveis para problemas complexos.
- **Hibridização**: combina global com local para melhores resultados.

Sempre valide seus resultados com múltiplos experimentos.

Próximos passos

- Pratique com os notebooks
- Aplique aos seus dados reais
- Teste validação cruzada

Leituras sugeridas

- Moré, J.J. (1978). LM Algorithm.
- Storn, R. (1997). DE Evolution.

Destaques da Literatura

- Holland, J.H. (1975). Adaptation in Natural and Artificial Systems.
- Goldberg, D.E. (1989). Genetic Algorithms.
- Ashyraliyev et al. (2009). Parameter estimation in systems biology.

Acesse o material complementar no Moodle/Classroom para exemplos avançados.

Obrigado!

Slides disponíveis em: <https://github.com/neylemkeunesp/bbs>

Ney Lemke — DBF-Unesp